

SymPy Bug Report

1 Bug 33: solveset Represents an Unsatisfiable Tangent Equation by Infinities

1.1 Minimal Reproducer

```
from sympy import Eq, I, S, solveset, symbols, tan

x = symbols("x")
sol = solveset(Eq(tan(x), I), x, domain=S.Complexes)
print("solution =", sol)
print("contains finite 0?", sol.contains(0))
```

1.2 Observed Behavior

```
solution = ImageSet(Lambda(_n, _n*pi + oo*I), Integers)
contains finite 0? False
```

SymPy returns a periodic image set whose elements are of the form $_n\pi + \infty i$. This is not a set of finite complex solutions to the equation $\tan(x) = i$. The recorded reproducer also evaluates the residual at $x = 0$ as $-1.0i$, confirming that an ordinary finite value is not being silently represented by the returned expression.

1.3 Expected Behavior

The equation $\tan(x) = i$ has no finite complex solution, so

$$\text{solveset}(\tan(x) = i, x, \mathbb{C}) = \emptyset.$$

The same conclusion holds for the paired singular target $\tan(x) = -i$.

1.4 Mathematical Explanation

For finite complex z ,

$$\tan(z) = -i \frac{\exp(2iz) - 1}{\exp(2iz) + 1}.$$

If $\tan(z) = i$, then

$$-i \frac{\exp(2iz) - 1}{\exp(2iz) + 1} = i \implies \frac{\exp(2iz) - 1}{\exp(2iz) + 1} = -1 \implies \exp(2iz) = 0.$$

The complex exponential is never zero at a finite complex argument, so the equation has no finite complex solution. For $\tan(z) = -i$, the same formula would require an infinite exponential value instead of a finite one. Thus the returned $oo*I$ -valued image set is not a harmless branch choice, a missing integer parameter, or an equivalent representation of a finite solution family; it represents a singular inverse value as if it were a valid complex solution.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant solver logic in `sympy/solvers/solveset.py`, specifically the complex trigonometric inversion helper around lines 467–473:

```
_invert_trig_hyp_complex
```

The traced call path starts with `solveset`, which rewrites the equation as $\tan(x) - i = 0$ in `_solveset`. The trigonometric equation detector then routes the expression to `_solve_trig`. Because the equation contains one trigonometric atom, `_solve_trig` calls `_invert`; the complex inversion path moves the additive $-i$ and inverts $\tan(x) = i$.

At that point, the same helper applies tangent’s periodic inverse pattern, effectively constructing $n\pi + \arctan(a)$ for a finite target a . For $a = i$, however, $\arctan(i)$ evaluates to $oo*I$ through the pure-imaginary branch in `sympy/functions/elementary/trigonometric.py`. The solver then embeds that non-finite inverse value into an `ImageSet` and returns it without a later finite-complex filter or an empty-preimage check.

This source-level behavior causes the mathematical failure because tangent’s singular target values i and $-i$ do not have finite complex preimages, but the inversion rule treats the infinite principal inverse as though it were a valid seed for the periodic solution family. The supported fix direction is to detect non-finite inverse values in the tangent and cotangent complex inverse branches, at least for $a = \pm i$, and return `EmptySet`; more generally, inverse image sets should be filtered to finite complex values.

1.6 Additional Failing Instances

The independently checked failure family consists of the two singular tangent targets i and $-i$. These are the values where the inverse tangent is non-finite rather than a finite complex seed for the usual $n\pi + \arctan(a)$ solution family. Both cases should therefore return the empty set over $\mathbb{C}$.

Input / Expression	SymPy behavior	Expected behavior
<code>solveset(tan(x) = i, x, C)</code>	<code>ImageSet(Lambda(_n, _n*pi + oo*i), Z)</code>	$\emptyset$
<code>solveset(tan(x) = -i, x, C)</code>	not <code>EmptySet</code> ; contains a non-finite inverse tangent value	$\emptyset$

The expected behavior is the same across the family because neither equation can be satisfied by finite complex x : one would force $\exp(2ix) = 0$, and the other would require the corresponding exponential term to become infinite.

1.7 Related Issues Assessment

A best-effort deduplication check found public material related only at the general level of trigonometric inversion and `ImageSet` representations for infinite families. It did not identify a public

issue, pull request, Stack Overflow post, or release-note entry for $\tan(x) = i$ returning an $oo * I$ -valued image set. The deduplication verdict is therefore that this is a new instance of a known failure mode: complex trigonometric inversion must reject non-finite inverse images, but this exact reproducible SymPy case appears previously unreported and remains individually actionable as an issue and a regression test.

1.8 Proposed SymPy Regression Test

```
import pytest
from sympy import Eq, I, S, solveset, symbols, tan

x = symbols("x")
CASES = [I, -I]

@pytest.mark.parametrize("target", CASES)
def test_tan_singular_targets_have_no_finite_complex_solution(target):
    assert solveset(Eq(tan(x), target), x, domain=S.Complexes) is S.EmptySet
```